

OOAD Ch9: 如何尋找類別與物件

主題重點：如何從需求中找出 [Class](#) 與 [Object](#)、[Jacobson](#) 的三大物件分類、[KRB Seven-Step Method](#) 七步分析法，以及 [Blockbuster](#) DVD 租片系統的分析案例。

課前導論

為什麼「找類別」很難？

- 老師一開始就強調：尋找類別與物件比想像中困難。
- 以課堂上的「供應商」分析測驗為例，真正能完整答對的人大約只有三分之一。
- 也就是說，這件事不是憑直覺亂抓幾個名詞就能做對，必須依賴一套有系統的方法。

物件導向分析與資料庫 ERD 的關係

- 老師提到，[Object-Oriented Analysis and Design](#) 的很多基礎，其實與資料庫的 [ERD](#) 很相似。
- 如果以前沒有修過資料庫，這部分通常會覺得比較吃力。
- 但兩者最大的差別在於：物件導向除了資料結構之外，還多了：
 - 行為 ([Method / Operation](#))
 - 繼承 ([Inheritance](#))
 - 多型 ([Polymorphism](#))

所以可以把 OO 分析看成是：

在 [ERD](#) 的基礎上，再加入行為與物件導向機制。

[Jacobson](#) 的三大物件分類

物件導向大師 [Jacobson](#) 認為，系統中的物件大致可以分成三大類。

1. Entity Objects (實體物件)

這類物件存在於使用者的真實世界中，是分析的核心。

(1) Concrete Objects (具體物件)

- 例如：[Employee](#)、[Product](#)、[Tool](#)
- 這些都是具體可辨識的東西

(2) Conceptual Objects (概念物件)

- 例如：[School](#)、[Membership](#)、[Strategy](#)
- 老師特別提醒：像「學校」其實是一種概念。
- 如果把校園建築、老師、學生全部抽離掉，單獨的「學校」並不是一個可摸可看的實體。
- 但因為它在業務世界中具有意義，所以仍然可能成為一個 object。

(3) Event and State Objects (事件與狀態物件)

- 例如：[Purchase](#)、[Delivery](#)、[Arrival](#)
- 老師舉例說，在購物台買完東西後，會產生一個「運送」的過程；這個過程本身也可以被視為一個物件。
- 也就是說，物件不一定只能是「人」或「東西」，也可能是事件或狀態轉變。

2. Interface Objects (介面物件)

- 用來封裝圖形使用者介面 (GUI)
- 或用來處理通訊協定 (communication protocols)
- 也包括控制 / 監控外部實體系統的物件

例如：

- 醫療行政系統中的某個 monitor
- 操作畫面上的表單物件
- 與外部設備互動的控制介面

3. Control Objects (控制物件)

- 用來放那些很重要、但不知道該歸到哪個 entity 下面的複雜方法
- 老師舉例：[ABS](#) (絕對值) 這種功能，掛在哪個實體物件之下都怪怪的，就可以獨立放進 control object

這三類的意義

[Jacobson](#) 的分類幫助我們和分析時，不會只想到資料表式的 entity，也能意識到：

- 有些物件代表真實世界資料
- 有些物件代表畫面或系統邊界
- 有些物件代表控制流程與複雜邏輯

[KRB](#) 七步分析法 ([KRB Seven-Step Method](#))

老師強調，[KRB](#) 方法的價值在於它提供了一套 [SOP](#)。

只要扎扎實實跟著 [SOP](#) 做，分析結果通常就會八九不離十。

這也是本章最重要的主軸。

1 步驟一：尋找候選類別 (Candidate Classes)

核心做法

- 盡可能把需求敘述中的 所有名詞 (nouns) 都抓出來
- 先不要急著刪，先大量蒐集

資料來源

- 客戶訪談 (Client Interviews)
- 系統文件與報表 (Documentation / Reports)
- 腦力激盪 (Brainstorming)

重點理解

這一步不是要你立刻判斷對錯，而是先建立「候選名單」。
也就是說：

- 先廣收
- 後面再過濾

2 步驟二：定義類別 (Define Classes)

把候選名詞撈出來之後，下一步要問：

它真的有資格成為 class 嗎？

老師提出兩個重要判準。

(1) 是否有真實世界識別碼？

- 一個 class 通常應該要能對應到某種 **unique identifier**
- 例如：
 - 學號
 - 車牌或 VIN
 - 身分證字號
 - 員工編號

老師也提醒：

- 名字、長相通常都不夠精確，不能當成真正可靠的 ID
- 如果一個名詞根本找不到獨立識別方式，往往就不是 class

(2) 是否有明確定義？

以 **Employee** 為例：

- 員工不一定只是「現在正在公司工作的人」
- 還可能包含：
 - 曾經工作過的人
 - 未來將加入的人

到底怎麼定義，要看企業的需求而定。

老師強調

不是所有名詞都是類別。

例如：

- Date**
- Color**

這些通常比較像是**屬性 (attribute)**，而不是具有獨立存在性的類別。

3 步驟三：建立關聯 (Establish Associations)

找出類別後，要開始思考它們之間的互動。

核心做法

- 尋找 **動詞 (verbs)**
- 把需求變成：

Object -> verb -> Object

例如：

- Student enrolls in Course**
- Customer buys Product**

畫線檢查法

KRB 建議一種很土法煉鋼但很有效的方法：

- 把所有類別列成清單
- 逐一檢查它和下面每個類別有沒有關聯
- 一條一條畫線確認

這樣做的目的不是漂亮，而是：

- 避免遺漏
- 避免只憑印象做分析

關聯句子的必要元素

一個完整的關聯描述至少要有：

1. 動詞
2. 多重性 (Multiplicity)

例如：

- 一個學生可以修很多課
- 一門課可以被很多學生修

4 步驟四：展開多對多關聯 (Expand Many-to-Many Associations)

這一步是整章最關鍵、也最容易做錯的地方。

問題本質

如果直接把兩個 class 用多對多 (M:M) 綁在一起，剛開始看起來很方便，但未來一定會出問題。

老師用「卡片與線」的資料庫比喻：

- 顧客卡片：Bo, Jo, Mo
- 產品卡片：Coffee, Donuts
- 若中間只是用線連起來，就等於程式裡用 vector / array 直接互指

多對多的三大缺點

(1) 沒地方放關聯本身的資料

例如購買關係除了「誰買了什麼」之外，還可能有：

- 數量
- 單價
- 金額
- 日期

這些資訊：

- 放在 Customer 不對
- 放在 Product 也不對
- 難道寫在線上？

(2) 容易資料重複與不一致

- 如果同一筆關係在兩邊都各自記錄
- 刪除或修改時就必須同步改兩邊
- 很容易因為 [Murphy's Law](#) 出現不一致

(3) 難以擴充

一旦未來要新增欄位，例如：

- 購買日期
- 負責業務員
- 折扣

用單純的互相指向結構會很難承接這些需求。

解決方法

把每條線換成「一張新卡片」。

也就是新增一個新的 class，例如：

- **Sale**
- **Enrollment**
- **Rental**

這個新 class：

- 代表原本那條關係本身
- 可以攜帶自己的屬性
- 把原本的 M:M 拆成兩個 1:M

關鍵觀念

這個步驟其實就是在做：

資料庫正規化 (Normalization)

也呼應了 [Ch8](#) 講的核心觀念。

5 步驟五：定義屬性 (Attributes)

做法

- 找出系統需要保留的資料欄位
- 這些欄位通常是名詞或形容詞
- 用來描述 object 的狀態與特徵

例如：

- Student 的姓名、學號、科系
- Product 的型號、價格、名稱
- Sale 的購買日期、數量、總額

重點

屬性是用來「描述」物件的，不應把本來應該獨立成 object 的東西硬塞成 attribute。

6 步驟六：正規化 (Normalization)

這一步主要是在防止初學者犯一個很常見的錯誤。

常見錯誤

看到一張報表之後，就直接建立一個超大 class：

- `SalesReport`
- 裡面同時有 Customer Name、Product Name、Quantity、Price...

這樣會把不同層次的資料全部混在一起。

正確做法

應該拆成合理的獨立類別，例如：

- `Customer`
- `Product`
- `Sale`

本質

不要被報表外觀騙了。

報表是輸出形式，不代表內部資料結構就應該長那樣。

7 步驟七：定義操作行為 (Operations / Behavior)

這是物件導向和傳統 ERD 最大的差異之一。

尋找 Operation 的技術

老師提到六種常見方法：

- By Inspection
- Basic CRUD
- Use Cases
- Statechart Diagram
- CRC Cards
- CRUD Revisited

找尋相似性 (Similarity)

當你發現不同物件有共同點時，就可以開始思考：

- 是否抽出共同父類別？
- 是否形成繼承架構？

例如：

- 不同學生個體之間有共同欄位與共同操作
- 可以抽象出同一個 `Student` class

Inheritance 與 Polymorphism

- **繼承 (Inheritance)**：找出共同結構與共同行為
- **多型 (Polymorphism)**：讓父類別的參考可以操作不同子類別

也就是說，最後不只是找資料，還要找：

- 這些物件能做什麼
- 哪些行為應該被共用
- 哪些地方應該保留彈性

🏠 老師延伸分享：SOP 與文化差異

老師在講 KRB 七步法時，特別延伸談到 SOP 的重要性。

麥當勞得來速故事

老師提到朋友在美國晚上 10 點去麥當勞：

- 當時大門沒鎖
- 但已經到了關門時間
- 店員堅持不在櫃台直接結帳
- 要求他先走出去，再開車到得來速重新點餐

老師要講的重點

亞洲人常會覺得：

- 這樣很死板

- 不知變通
- 幹嘛不直接賣

但老師認為，在軟體工程裡：

SOP 不是笨，而是避免災難。

如果分析時自作聰明、跳步驟、不肯窮舉、不肯好好定義類別，最後就很容易做出：

- 難維護
- 難擴充
- 一改就爆炸

的系統。

📺 實作案例：Blockbuster DVD 租片系統分析

老師最後給了一個非常典型的分析練習。

需求一：影片計價邏輯

新片

- 租一次 **\$3**
- 可租 **1 天**
- 逾期每多一天加 **\$2**

舊片

- 租一次 **\$2**
- 可租 **3 天**
- 逾期每多一天加 **\$1**

兒童片

- 租一次 **\$2**
- 可租 **5 天**
- 逾期每多一天加 **\$1**

此外：

- 一部電影會有很多實體拷貝片可以租

需求二：常客積點優惠

這個規則老師特別提醒：經常會改，所以設計時要保留彈性。

- 新片：每租一片得 **2.0** 點
- 舊片：每租一片得 **1.0** 點
- 兒童片：每租一片得 **0.5** 點

這表示在設計時，不能把規則寫死在一堆 if/else 裡，否則未來改制度時會很痛苦。

需求三： `print_statement()`

這個 method 需要能夠印出：

1. 過去租過的片子清單
2. 總消費金額
3. 常客積點總數

從需求可推測的類別

這個案例很適合練習 KRB 七步法。可能的主要 classes 包括：

- **Customer**
- **Movie**
- **Rental**
- **PriceStrategy** 或其他計價策略類別
- **Copy** (若要區分電影作品與實體片)

分析重點

1. **Customer** 與 **Movie** 不應直接多對多

- 因為租片這件事本身有：
 - 租借天數
 - 價格
 - 點數
 - 可能還有租借日期

所以中間應該有 **Rental** 這種關聯物件。

2. 計價規則經常變動

這暗示了：

- 需要保留策略彈性
- 可能適合使用 **Strategy** 思維
- 不應只用巨大條件判斷寫死

3. `print_statement()` 放哪裡？

題目特別要求思考 `print_statement()` 屬於哪個 class。

合理上它通常會和：

- [Customer](#)
- 或某個專門負責報表輸出的物件

有關，因為它是在列印特定客戶的租借歷史、費用與點數。

✔ 本章核心觀念總結

1. 找類別不是憑感覺，而是要靠方法。
2. [Jacobson](#) 把物件分為 **entity / interface / control** 三大類。
3. KRB 七步法的核心價值是 **SOP** 化分析流程。
4. 不是所有名詞都能當 **class**，要檢查識別性與定義。
5. 關聯一定要搭配 **動詞** 與 **多重性** 來思考。
6. **多對多** 必須展開，中間建立新的關聯物件，這其實就是正規化。
7. 物件導向比 ERD 多出的關鍵，是 **behavior**、**inheritance**、**polymorphism**。
8. 面對像 [Blockbuster](#) 這種案例，除了找 class，更要思考哪些規則未來會變、哪裡該保留彈性。

🔗 Related Notes

- 📖 [School/OOAD_Object-Oriented Analysis and Design](#)
- [School/OOAD/CH8_OOAD_ERD_and_Normalization](#)
- [School/OOAD/QA5_OOAD_Class_Notes](#)